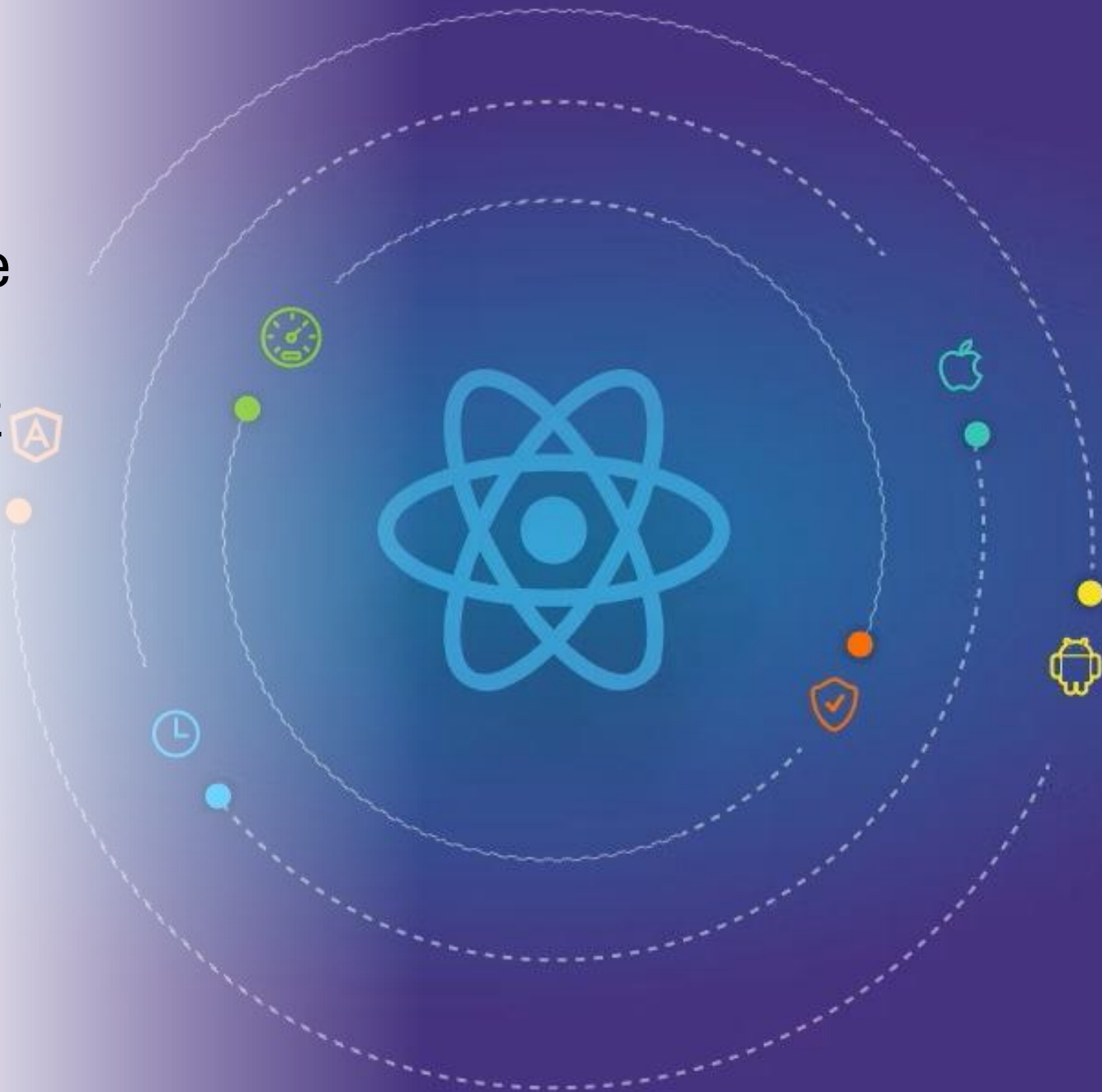


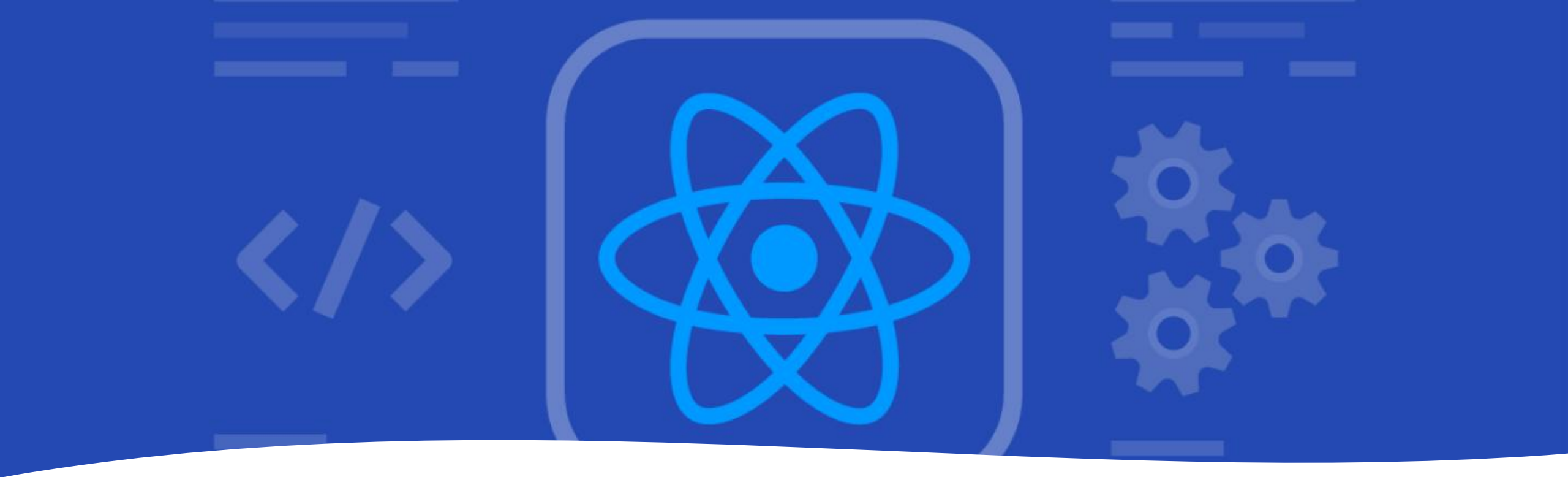
Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

13. A osztály

Tanárok: Dr. Koncz Adrienn és
Simon Martin

2025.09.11.





Mi az a React Native?

- A React Native egy nyílt forráskódú keretrendszer, ami azt jelenti, hogy bárki számára ingyenesen elérhető.
- A szoftverfejlesztésben a keretrendszer egyfajta platformot jelent az alkalmazások fejlesztéséhez.

Mi az a React Native?

- Olyan erőforrásokat tartalmaz, mint előre elkészített komponensek, könyvtárak és hivatkozási anyagok.
- Hasonlóan ahhoz, ahogy egy ház vázszerkezete adja az alapot, a React Native biztosítja az alkalmazás alapszerkezetét.
- A fejlesztők ezután testre szabhatják a natív modulokat igényeik szerint, ahelyett hogy az alkalmazást teljesen a nulláról kellene felépíteniük.

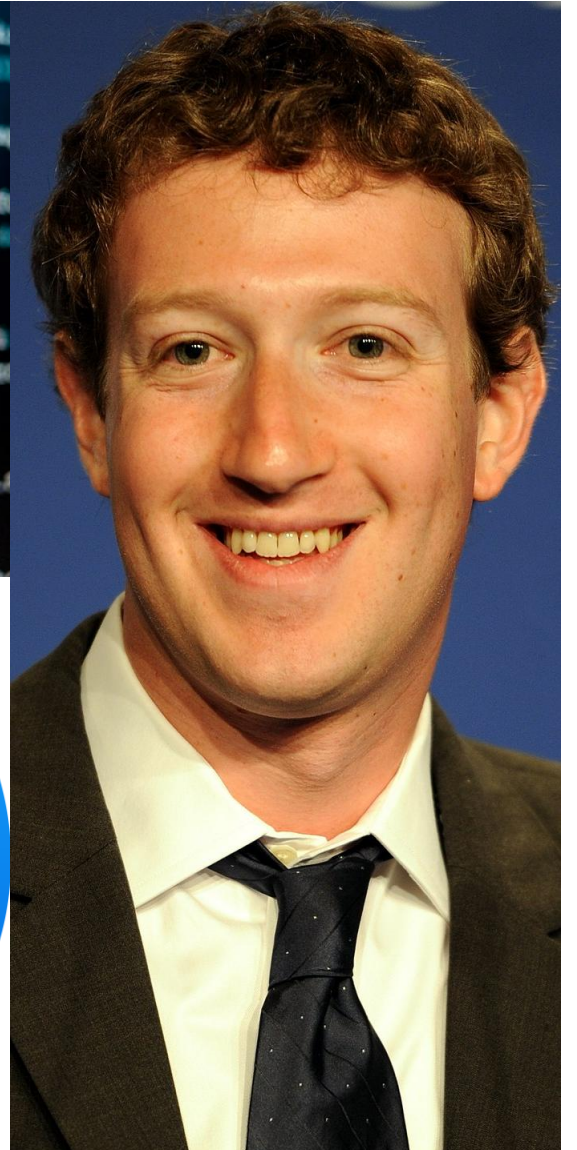
```
import React, { Component } from 'react';
import StyleSheet, Text, View, Image,
  ScrollView, TouchableOpacity
  from 'react-native';
import { connect } from 'react-redux';
import {
  fetchAllAmicis } from '../actions/amiciActions';
import AmiciWikiSingle from './AmiciWikiSingle';

class WikiMain extends Component {
  componentWillMount() {
    this.props.fetchAllAmicis();
  }

  render() {
    return (
      <View style={styles.container}>
        <ScrollView
          ref={(scrollView) => { _scrollView = scrollView;
            automaticallyAdjustContentInsets={false}
            scrollEventThrottle={200}}>
          {this.props.amicis ?
            this.props.amicis.map(function(amic, i){
              return (
                <AmiciWikiSingle key={i} amici={amic} />
              )
            }) : <Text>'</Text>
          }
        </ScrollView>
      </View>
    );
  }
}
```

A React Native története

- Amikor a Facebook először döntött úgy, hogy szolgáltatásait mobil eszközökön is elérhetővé teszi, nem a legtöbb nagy technológiai szereplő által választott utat követte, vagyis nem natív alkalmazást épített, hanem egy HTML5-alapú mobilweboldal mellett tette le a voksát.
- Ez a megoldás azonban nem állta ki az idők próbáját: a felhasználói felület és a teljesítmény terén is sok kívánnivalót hagyott maga után.
- 2012-ben Mark Zuckerberg nyíltan elismerte: *„a legnagyobb hibánk cébként az volt, hogy túl nagy tétet tettünk a HTML-re a natív megoldások helyett.”*

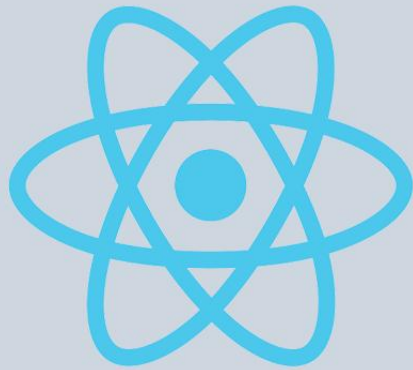




- Nem sokkal később, 2013-ban a Facebook egyik fejlesztője, Jordan Walke áttörő felfedezést tett: kidolgozott egy módszert, amellyel JavaScript segítségével lehetett UI-elemeket generálni iOS-alkalmazásokhoz.
- Ez igazi lavinát indított el, és a Facebook egy külön Hackathont szervezett, hogy kiderítse, milyen mértékben lehet mobilfejlesztést végezni az addig főként webes környezetben használt JavaScript-megoldásokkal.
- Így született meg a React Native. Kezdetben kizárólag iOS-re fejlesztették, ám a Facebook hamarosan Android-támogatással is kibővítette, majd 2015-ben nyílt forráskódú keretrendszerként tette elérhetővé.

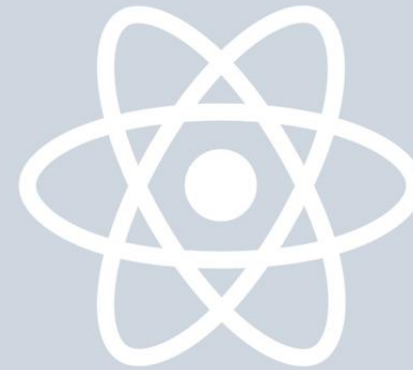


A hackathon olyan rendezvény, aminek célja, hogy üzletemberekből, szoftverfejlesztőkből és más szakemberekből álló csapatok egy meghatározott témában rövid időn belül működő informatikai megoldást készítsenek.



ReactJS

VS



React Native

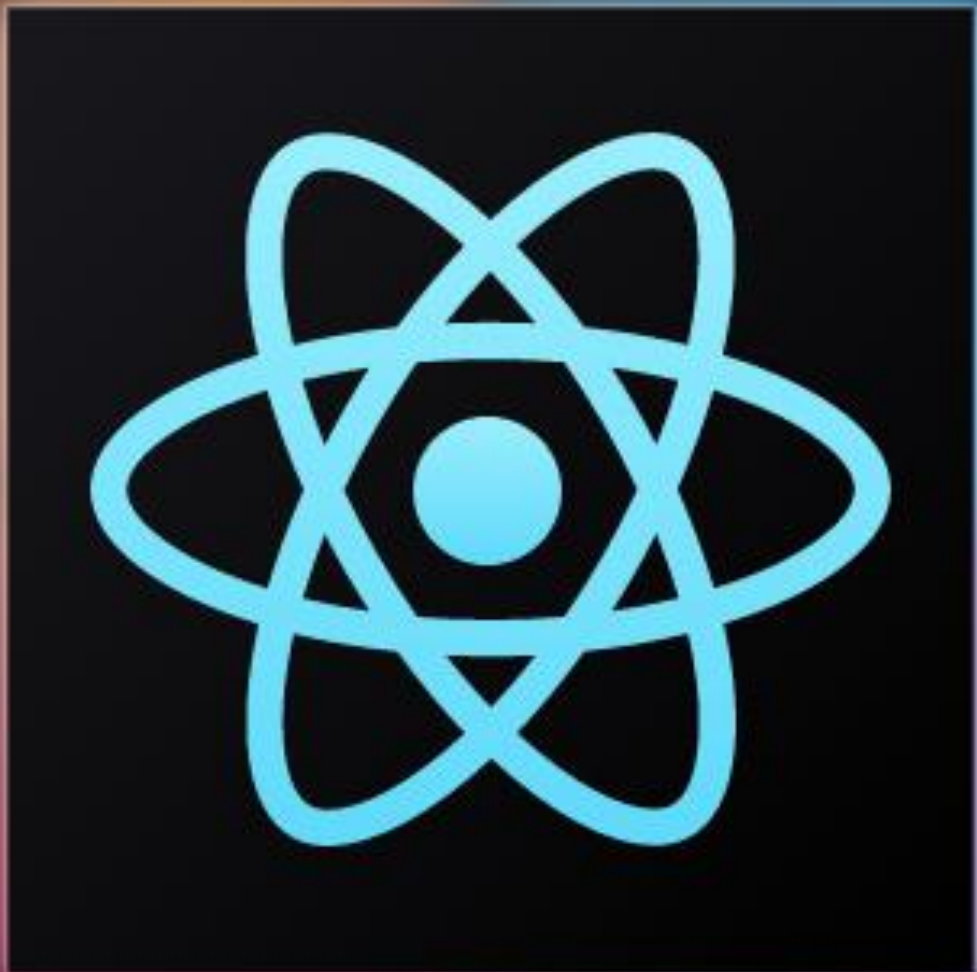
React és React Native

- A React egy JavaScript-könyvtár, más néven ReactJS vagy React.js.
- A React Native-hez hasonlóan fejlesztői komponenseket biztosít, például előre megírt kódokat és konfigurációkat.
- A különbség az, hogy míg a React webes komponenseket használ, addig a React Native natív komponenseket.
- A két technológia együtt használva kiegészíti egymást.



Mi az az Expo React Native?

- Az Expo egy keretrendszer JavaScript- és React Native-alkalmazások fejlesztésére.
- Könyvtárakat, valamint kifejezetten a React Native fejlesztéséhez és teszteléséhez készült szolgáltatásokat és eszközöket tartalmaz.
- Az Expo célja, hogy leegyszerűsítse az univerzális alkalmazásfejlesztést a webre és mobilplatformokra, például Androidra és iOS-re.



Miért használjuk a React Native-ot?

- A React Native-ot hosszú ideig kereskedelmileg életképtelennek tartották.
- Nem fejlesztették vagy támogatták eléggé ahhoz, hogy „natív jellegű” alkalmazásokat hozzon létre.
- De az idők megváltoztak. A React Native egyre népszerűbb, egyre nagyobb a közösség támogatása, és egyre nagyobb piaci részesedést szerez.
- A React Native segítségével egyre könnyebb és egyszerűbb zseniális alkalmazásokat írni – és a világ ezt észreveszi.
- És a dokumentációja is nagyon fejlett már, így egy kezdő is önszorgalomból képes elsajátítani a megfelelő tudást. <https://reactnative.dev/>

Népszerű React Native app-ok

- Facebook
- Instagram
- WhatsApp
- Tesla
- Amazon
- eBay
- TikTok
- Netflix
- Spotify
- Messenger
- Airbnb
- Szallas.hu
- Wolt
- Foodora



Használhatjuk vele a következő nyelveket

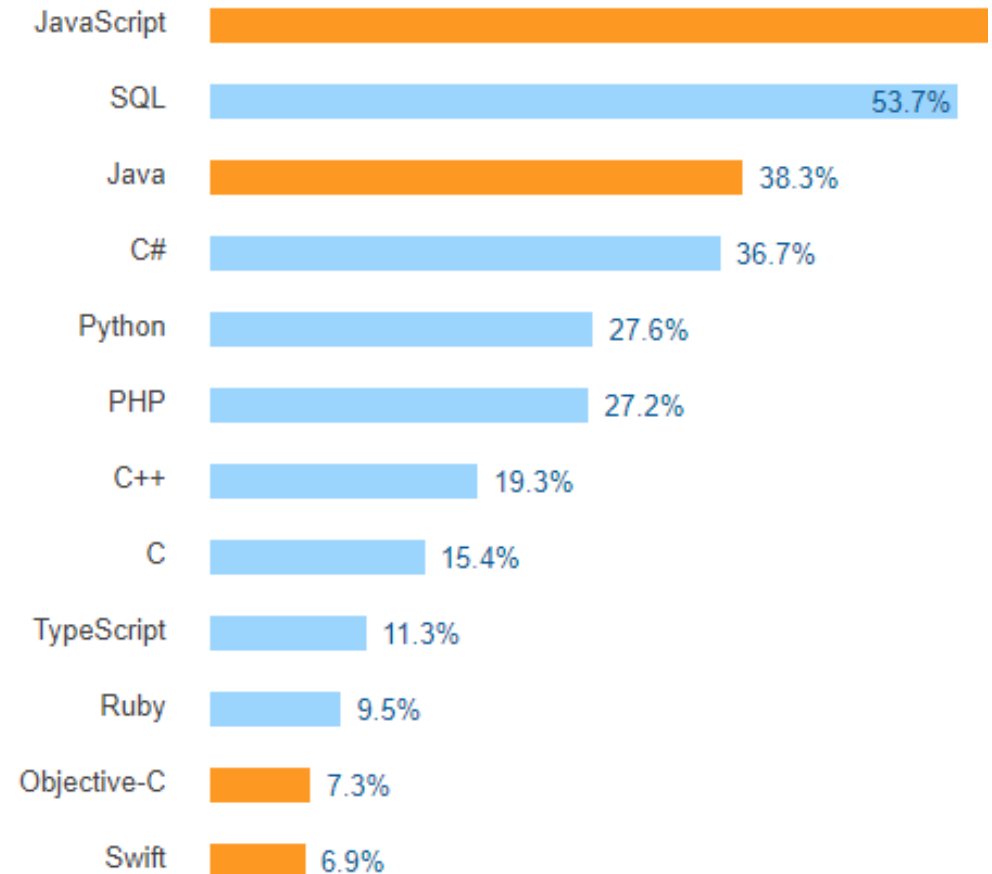
- Kevésbé speciális tudásra van szükség a React Native alkalmazások létrehozásához.
- Aki tud JavaScriptben programozni, esetleg kifejezetten ReactJS-ben, annak könnyű dolga lesz.
- A cégeknek is a JS-fejlesztőket egyszerűen könnyebb megtalálni, mint a Java- vagy Swift-tudással rendelkező fejlesztőket.

Programming Languages

% of This Category

% of All Respondents

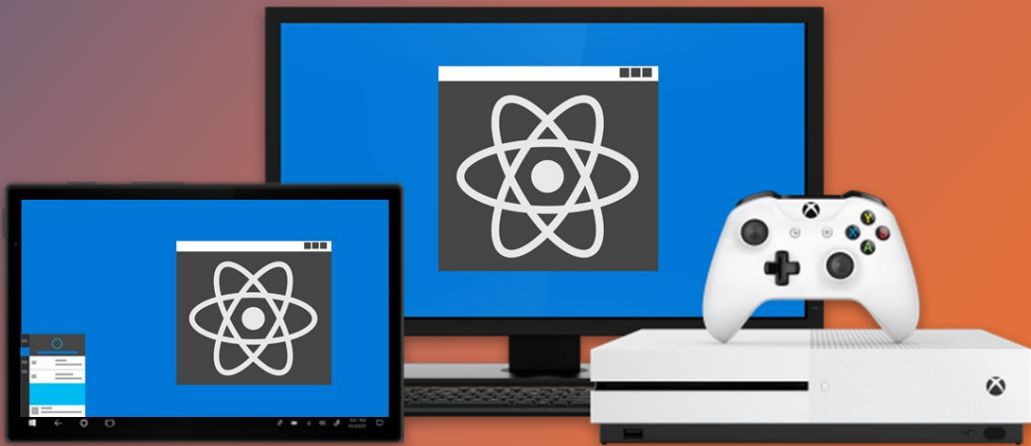
% of Professional Developers





React Native on tvOS

by Neil Kimmett



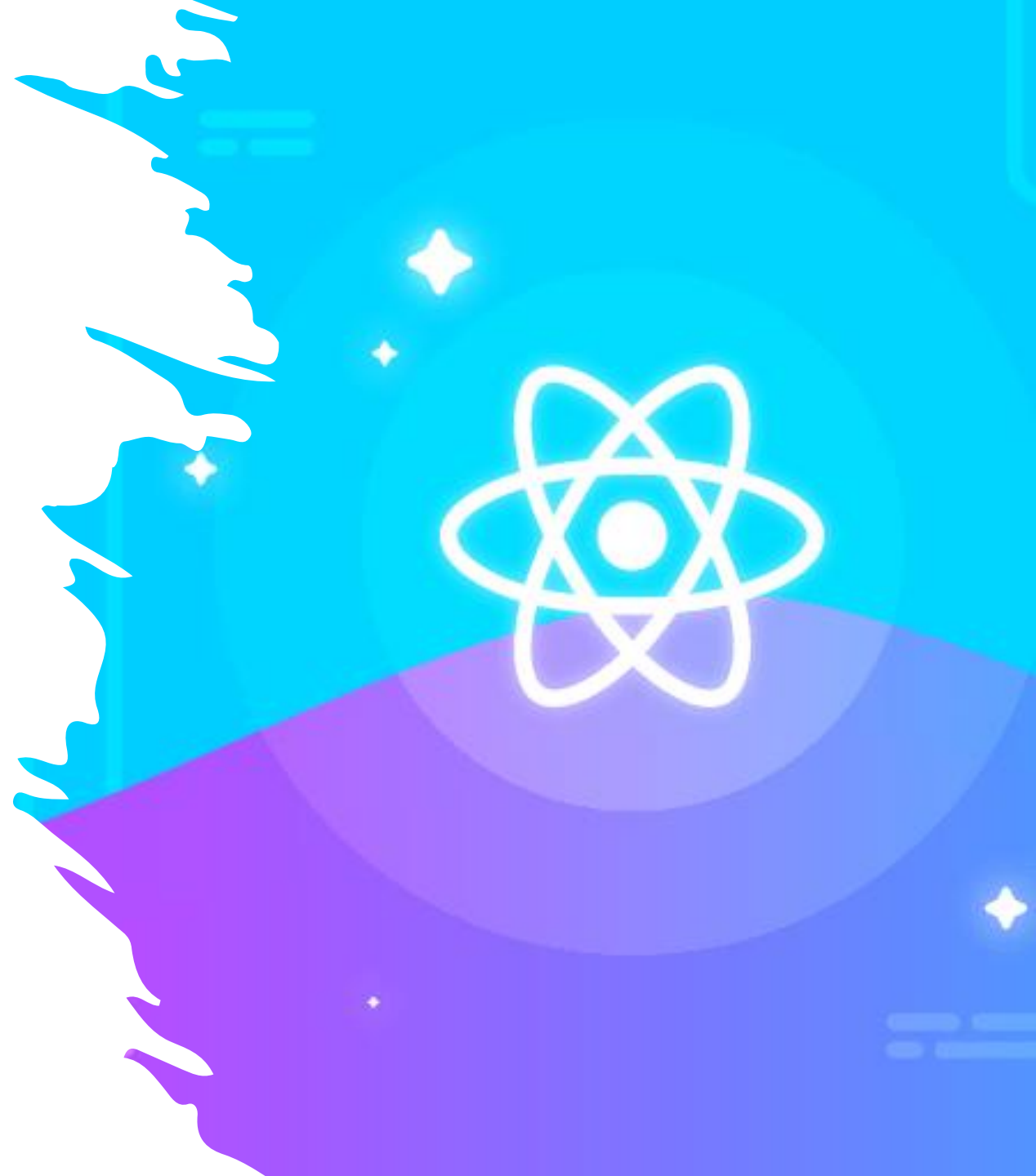
További alkalmazása a React Native-nek

- Android TV, macOS, tvOS, Web, Windows és UWP (Universal Windows Platform).
- A Facebook mellett Android és iOS alkalmazások fejlesztésére használják a Microsoft és a Shopify oldalakon is.
- Virtuális valóság-alkalmazások fejlesztésére is használják az Oculusnál.

A React Native főbb jellemzői

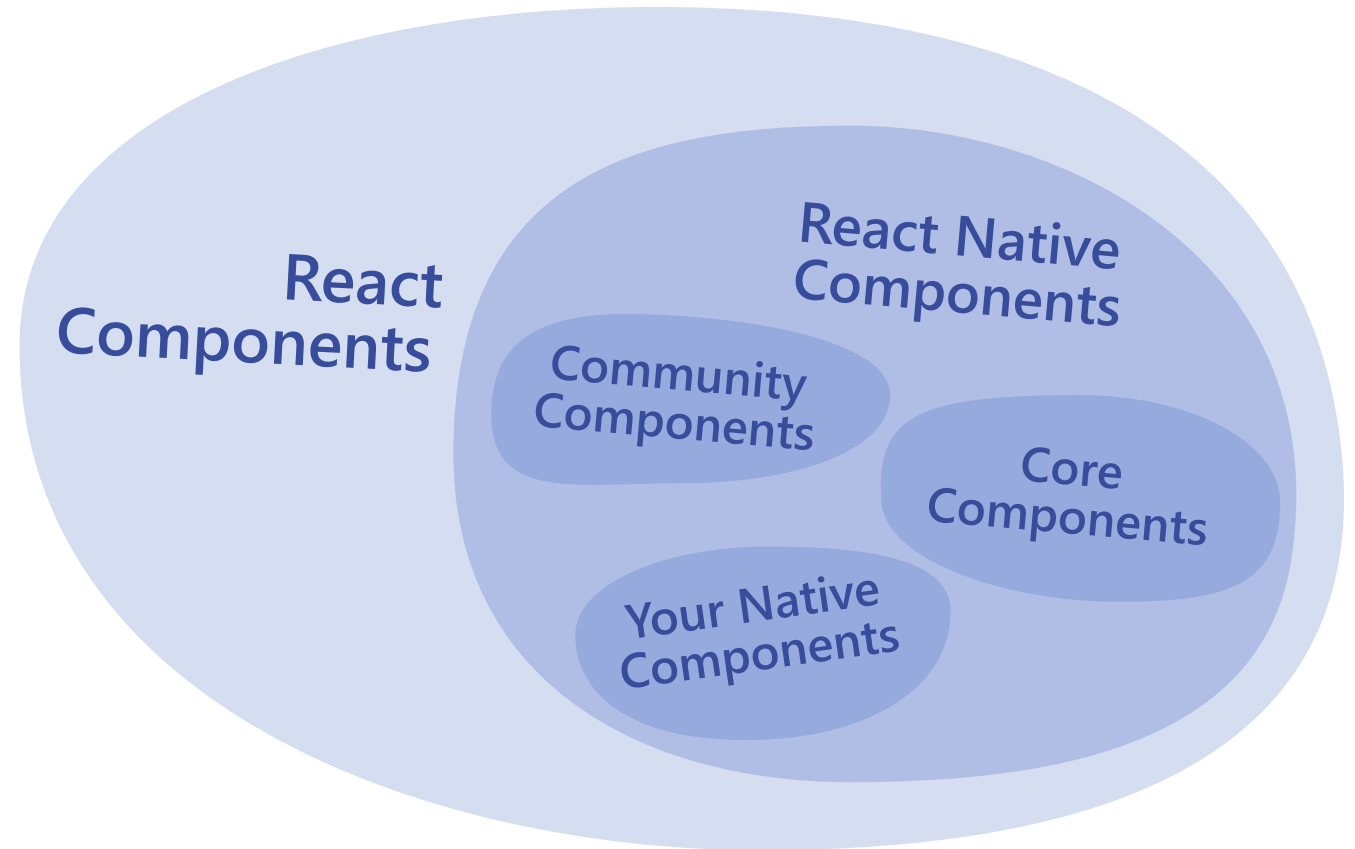
- **Komponens alapú architektúra:** A React Native ugyanazt a komponensalapú megközelítést követi, mint a React, ahol az alkalmazás különböző részei különálló komponensekbe szerveződnek.
- **Hot Reloading:** Lehetővé teszi a kódváltoztatások azonnali tesztelését anélkül, hogy újra kellene indítani az alkalmazást.
- **Cross-platform:** Ugyanaz a kód futtatható Androidon és iOS-en, de lehetőség van platformspecifikus megoldásokra is, ha szükséges.

Ez a keretrendszer különösen népszerű a startupok és azok körében, akik gyorsan szeretnének natív mobilalkalmazásokat fejleszteni, mert nagyfokú újrafelhasználhatóságot kínál a kód számára a különböző platformokon.



Mit jelent, hogy a React Native architektúrája komponens alapú?

- A komponensalapú architektúra a React Native egyik alapvető jellemzője, amely a felhasználói felület felépítését egyszerűsíti azáltal, hogy az alkalmazást kisebb, újrahasznosítható komponensekre bontja.
- Ezek a komponensek különálló, önálló egységek, amelyek egy adott funkciót vagy UI-részt valósítanak meg.



Mit jelent ez pontosan?

- **Újrafelhasználhatóság:** Minden komponens egy önálló, újrahasznosítható egység, amit különböző helyeken fel lehet használni. Például, ha egy gombot egyszer megírsz komponensként, azt bármelyik képernyőn használhatod anélkül, hogy újra kellene írnod.
- **Modularitás:** Az alkalmazást kisebb egységekre, komponensekre lehet bontani, amelyek könnyebben kezelhetők és fejleszthetők. Minden komponens saját logikával és megjelenéssel rendelkezik, így a komplex alkalmazásokat is átláthatóbbá teszi.
- **Könnyebb karbantarthatóság:** Mivel a komponensek különálló egységek, egy adott komponens módosítása nem feltétlenül befolyásolja az egész alkalmazást, így könnyebbé válik a hibakeresés és a karbantartás.
- **Hierarchikus felépítés:** A React Native alkalmazások felépítése hierarchikus, azaz az egyes komponensek tartalmazhatnak más komponenseket is. Például egy alkalmazásban lehet egy Header (fejléc) komponens, amelyben lehet egy Logo komponens és egy Menu komponens.

```
import React from 'react';
import { Text, View } from 'react-native';

const HelloWorld = () => {
  return (
    <View>
      <Text>Hello, World!</Text>
    </View>
  );
};

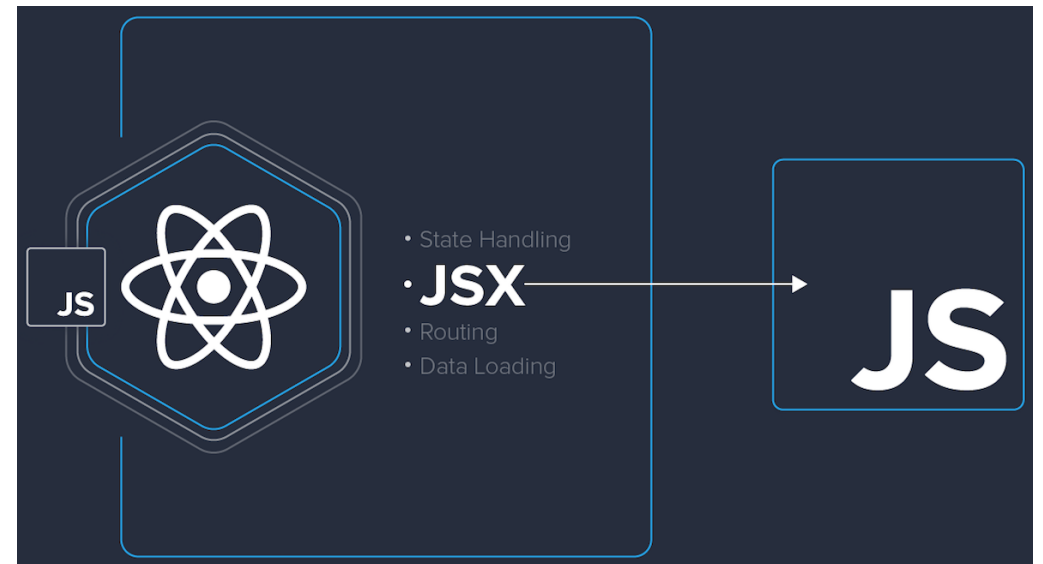
export default HelloWorld;
```

Ebben a példában a HelloWorld egy egyszerű komponens, amely egy szöveget jelenít meg. Ezt a komponenst bárhol újra felhasználhatjuk az alkalmazásban.

<https://reactnative.dev/docs/components-and-apis>

Mi az a JSX?

- A JSX (JavaScript XML) egy szintaxis, amelyet a React és a React Native használ, hogy JavaScriptben HTML-szerű kódot írassunk.
- A JSX segítségével sokkal intuitívabb módon lehet komponenseket és felhasználói felületeket definiálni, mivel a JavaScript kódon belül közvetlenül HTML-szerű elemeket írhatunk.



Példa a JSX-re

- Ebben a példában a View és Text elemek JSX-ben vannak megírva, de a motor végül ezeket natív kóddá alakítja Androidon és iOS-en.
- A View a natív Android/iOS "container" elemnek, a Text pedig a natív szövegelemnek felel meg.
- A React Native nem a JSX kódot futtatja közvetlenül. A JSX kódot először JavaScript kóddá alakítják át, majd a React Native ezt a kódot natív komponensekké (pl. iOS vagy Android UI elemekké) alakítja át, így az alkalmazás natív élményt nyújt.

```
import React from 'react';
import { Text, View } from 'react-native';

const MyComponent = () => {
  return (
    <View>
      <Text>Hello, JSX!</Text>
    </View>
  );
};

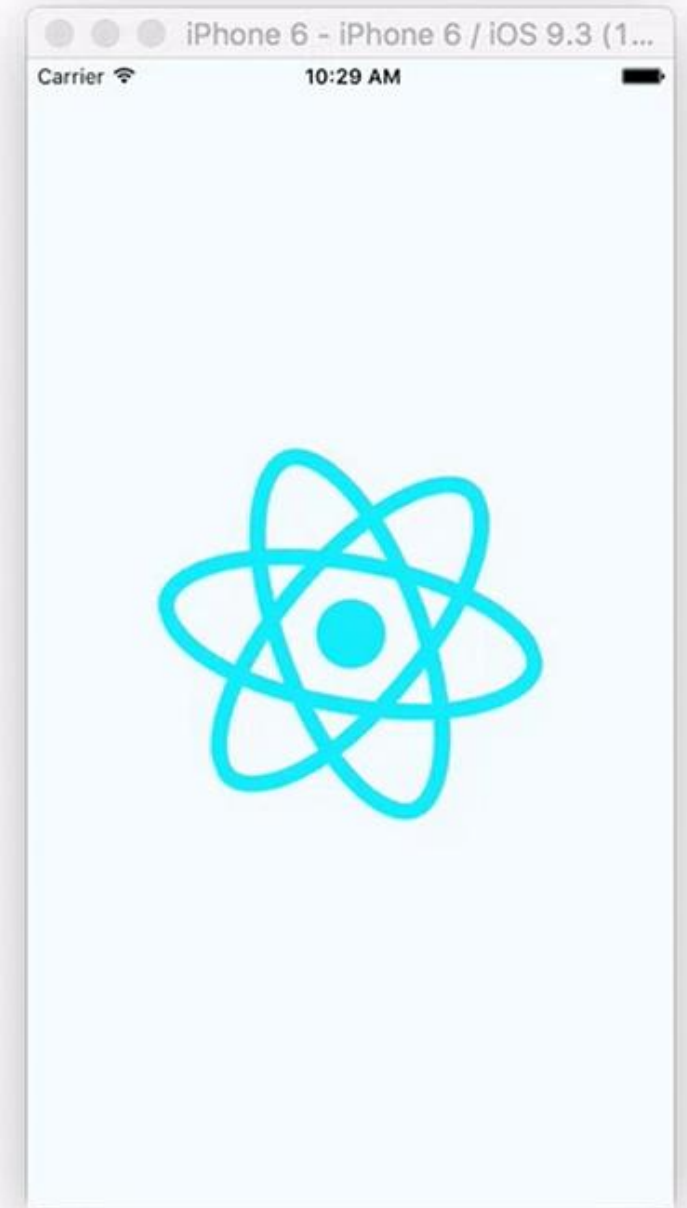
export default MyComponent;
```

- A React Native weboldala: <https://reactnative.dev/>
- Az expo.dev oldala: <https://expo.dev/>

Az oldalhoz tartozó mobilapp → Expo Go
(App Store-ból és Google Play-ből
letölthető)



Expo Go



Hogyan fejlesszünk (kódoljunk) gépen?

- Amire szükség van a környezet, kódolás kialakításához:
 - Node.js
 - Android Studio, amiből csak az emulatort kell használni
 - Expo projekt létrehozása
 - Böngésző
- Kódszerkesztéshez a VS Code-ot fogjuk használni, vagy az expo.dev weboldalt

Első Expo React Native projektünk létrehozása

1. Parancssort (cmd) nyissuk meg
2. Hozzunk létre saját részre egy mappát, vagy ha van már saját mappátok, akkor abban egy reactnative vagy mobil nevű mappát, amiben majd létrehozzuk a projekteket.
3. A mobil projektes mappába navigáljunk az cmd-ben, és adjuk ki a következő parancsok valamelyikét:
 - `npx create-expo-app@latest`
 - Itt fogja kérni, hogy nevezzük el a projektünket. FONTOS! Ékezetes betűt ne használjunk a projektek elnevezéséhez.
 - `npx create-expo-app my-app`
 - Itt a my-app helyet bármire elnevezhetjük rögtön a projektet
 - `npx create-expo-app my-app --template bare-minimum`

Mi a különbség a kettő között?

`npx create-expo-app my-app`

`npx create-expo-app my-app --template bare-minimum`

`npx create-expo-app my-app`

- Ez jelenleg a default template, ha ezt választjuk a projektet .tsx kiterjesztésű fájlokkal hozza létre
- A fő fájl, amit szerkesztenünk kell: `app/(tabs)/index.tsx`

`npx create-expo-app my-app --template bare-minimum`

- Ezzel viszont .js kiterjesztésű fájlokat hoz létre a projektben
- A fő fájl, amit szerkesztenünk kell: `App.js`

Alternatív megoldás

- <https://expo.dev/> oldalon regisztrálni kell



Snacks

+ New Snack



Alap React Native komponensek

- **View:** A felhasználói felület (UI) felépítésének legalapvetőbb komponense.
- **Text:** Szöveg megjelenítése valósítható meg vele.
- **ScrollView:** Egy görgethető konténer, amely több komponenst és nézetet is tartalmazhat.
- **Image:** Képeket tudunk vele megjeleníteni.
- **StyleSheet:** Stílus komponens, ami nagyon hasonlít a CSS-hez.
- **TextInput:** Lehetővé teszi a szöveg bevitelét a billentyűzet segítségével.
- **Button:** alapvető gombkomponens az érintések kezelésére.

Készítsük el az első kódunkat!

```
import React from 'react';
import { Text, View } from 'react-native';

const App = () => {
  return (
    <View>
      <Text>Hello!</Text>
    </View>
  );
};

export default App;
```

Mit csinál ez a kód?

import React from 'react';

- Ez importálja a React könyvtárat.
- A React szükséges ahhoz, hogy komponenseket hozzassunk létre, és kezelhessük azokat a szabályokat, amelyek meghatározzák, mit és hogyan jelenítsen meg az alkalmazás.

import { Text, View } from 'react-native';

- Itt a React Native beépített komponenseit importáljuk.
- View: olyan, mint egy „doboz” vagy „konténer”, amely más komponenseket (pl. szöveget, képet) tartalmazhat.
- Text: szöveg megjelenítésére szolgáló komponens.

Mit csinál ez a kód?

const App = () => { };

- Létrehozunk egy saját funkcionális komponenst App néven.
- Ez egy arrow function (=>), amely visszaadja azt, amit a felhasználónak látni kell.

return (....);

- Ez a rész megmondja, mit kell visszaadnia (renderelnie) a komponensnek.

Mit csinál ez a kód?

```
<View>
```

```
  <Text>Hello!</Text>
```

```
</View>
```

- A JSX szintaxist látjuk (JavaScript XML), amely lehetővé teszi, hogy HTML-szerű kódot írjunk a JavaScriptben.
- `<View>`: tartalmazza az összes belső elemet.
- `<Text>Hello!</Text>`: egy szöveges elem, ami a képernyőn megjelenik: „Hello!”

export default App;

- Ez az utasítás alapértelmezett exportot készít.
- Így más fájlokban egyszerűen importálhatjuk a komponenst:
- `import App from './App';`

Gyakorló feladat – Text, View, ScrollView

- Készítsünk a Text, View és ScrollView komponensek felhasználásával egy egyszerű screent.
- A feladathoz használjuk a gyak1.txt-ben lévő szöveget





Ez így elég unalmas
és csúnya 😞

- Adjunk hozzá egy minimális stílust, amihez használjuk a StyleSheet komponenst és készítsünk stílusosztályokat a komponensekhez.